

0. 방향성

이 장은 하드웨어 부분의 정점이다.

내장식 프로그램(Stored Program)에 대해 이야기 할 것

컴퓨터 과학의 기초인 폰 노이만 아키텍처 설명할 것

ALU와 레지스터 및 메모리 시스템을 이용해 어떻게 Hack 플랫폼을 구현하는지를 볼 것이다.

1. 내장식 프로그램

내장식 프로그램 개념은 1930년대 몇 명의 수학자들이 독립적으로 고안해 낸 것이다.

기본적인 아이디어는 컴퓨터는 정해진 명령어 집합들을 실행하는 고정된 하드웨어 플랫폼에 기반한다. 동시에 컴퓨터가 실행하는 명령어들은 기본 블록이 되어 무한히 복잡한 프로그램으로 조합된다. 이 프로그램들은 하드웨어에 내장되지 않는다. 그 대신 프로그램의 코드는 컴퓨터 메모리상에 바로 데이터처럼 저장되고 연산되고, 이것을 소프트웨어라고 부른다.

2. 폰 노이만 구조

내장식 프로그램 개념을 잘 보여주는 것은 2가지 있다. : 보편 튜링 기계/폰 노이만 기계

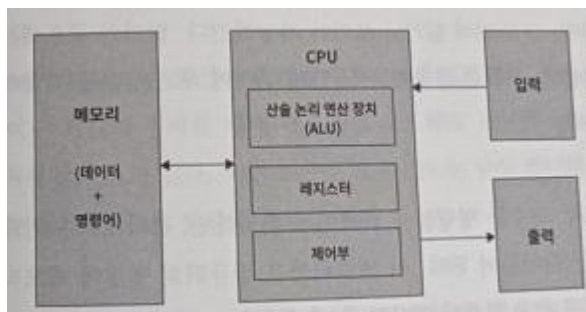
-튜링기계는 주로 컴퓨터 시스템의 논리적 기초를 분석하는 데 쓰인다.

-폰 노이만 기계는 현대의 거의 모든 컴퓨터 플랫폼의 개념적/실질적 구조이다.

폰노이만 구조는 중앙처리장치(CPU)를 기반으로 메모리 장치와 통신 /

입력장치에서 데이터 받고, 출력 장치에 데이터를 보내는 일을 한다.

메모리는 컴퓨터가 조작하는 데이터/ 컴퓨터가 수행하는 명령어 저장



3. 메모리

폰 노이만 기계의 메모리는 데이터/명령어 2가지의 정보를 저장한다.

2가지 정보는 무작위 접근 구조(random-access structure)로 2진수로 저장된다.

*명령어 메모리

컴퓨터 연산의 각 단계마다 CPU는 명령어 메모리에서 워드 하나를 인출(fetch)하고 해석한 후에 정해진 명령을 수행하고 다음에 실행할 명령어가 무엇인지 찾아낸다.

명령어 메모리에 상주하는 명령어는 기계어라 불리는 미리 정의된 형식으로 작성된다.

4. 중앙 처리 장치

CPU가 작업을 수행하는 데, ALU/레지스터/제어장치가 그것이다.

1) ALU

2) 레지스터

CPU는 간단한 계산을 빠르게 수행하도록 설계된다.

계산 성능을 올리려면 이런 계산의 결과를 매번 메모리에 로드하고 저장하기보다는, CPU 근처에 저장할 필요가 있다.

3) 제어 장치

컴퓨터 명령어는 폭이 16,32, 또는 64비트인 2진 코드로 표현된다.

명령어가 실행되려면 일단 명령어를 해석해서 하드웨어 장치가 명령을 어떻게 실행해야하는지 알아내야하고 이를 전달해야한다.

명령어 해석은 제어장치에서 이루어진다.

CPU연산은 명령어(워드)를 메모리에서 인출/해석/실행하며 다음 명령어를 인출하는 것으로 반복된다.

5.레지스터

레지스터는 CPU 칩 내부에 물리적으로 위치하므로 거의 바로 접근이 가능하다.

레지스터는 소수만 있기 때문에 조작할 레지스터를 지정하는데 몇 개의 비트만 필요하고, 명령어 길이도 더 짧게 된다.

1). 데이터 레지스터

-CPU의 단기 기억 메모리 기능을 한다.

2). 주소 지정 레지스터

-CPU는 데이터를 읽고 쓰기 위해 계속해서 메모리에 접근해야 한다. 이 연산마다 메모리 워드를 지정해야 한다.(주소 필요) 이 주소를 담는 레지스터이다.

*프로그램 계수기 레지스터(Program Counter Register)

프로그램을 실행할 때, CPU는 명령어 메모리에서 인출해야할 다음 명령어의 주소를 항상 알고 있어야 한다. 이를 프로그램 계수기 레지스터(PC)가 저장한다.

PC: 명령어 메모리에서 인출할 명령어의 주소

6. 입력과 출력

입출력 장치에는 스크린, 키보드, 프린터, 스캐너, 네트워크 인터페이스 카드, CD-ROM등이 있다.

컴퓨터 과학자들이 입출력 장치들을 컴퓨터에서 완전히 똑같이 처리할 수 있는 기법들을 고안했다. 기법 중 가장 간단한 것은 메모리 매핑IO이다.

기본 아이디어: IO장치를 에뮬레이션해서, CPU에게 그 장치를 마치 일반적인 메모리 세그먼트처럼 보이도록 하는 것이다.

각 IO 장치마다 메모리 내에 전용 영역이 할당되어 메모리 맵 역할을 하게 된다.

하드웨어 측면에서 IO 장치들은 메모리 장치와 유사한 인터페이스를 제공해야한다.

소프트웨어 관점으로는 각 IO장치에 통신 규약을 정의해서 프로그램이 올바르게 접근할 수 있도록 해야 한다.

메모리 매핑 IO 아키텍처는 중요하다. 제품의 특성과 무관하게 CPU나 전체 플랫폼을 설계할 수 있기 때문이다.

우르는 새로운 IO 장치를 컴퓨터에 연결하고 싶을 때마다 그저 그 장치를 새로운 메모리 맵에 할당하고 시작 주소를 메모해 두기만 하면 된다.

7. 핵 하드웨어 플랫폼 명세

핵 CPU는 ALU와 데이터레지스터 D/주소 지정 레지스터 A/ 프로그램계수기(PC) 로 구성된다.

우리의 컴퓨터 아키텍처에서 프로그램 계수기의 출력에서 나온 선은 ROM칩의 주소 입력에 연결된다. ROM 칩은 ROM[PC]를 출력한다. 이 값이 현재 명령어이다.

한 클록 주기동안 아래와 같이 동작한다.

*실행(EXECUTE)

명령어는 여러 칩에 동시 전송된다..

명령어가 주소 명령어라면 A레지스터로

명령어가 계산 명령어라면 D레지스터로

*인출(Fetch)

다음에 인출할 명령어는 현재 명령어의 점프 비트 및 ALU 출력에 따라 결정된다.

점프가 이루어지는 지는 이 두 값을 봐야한다.

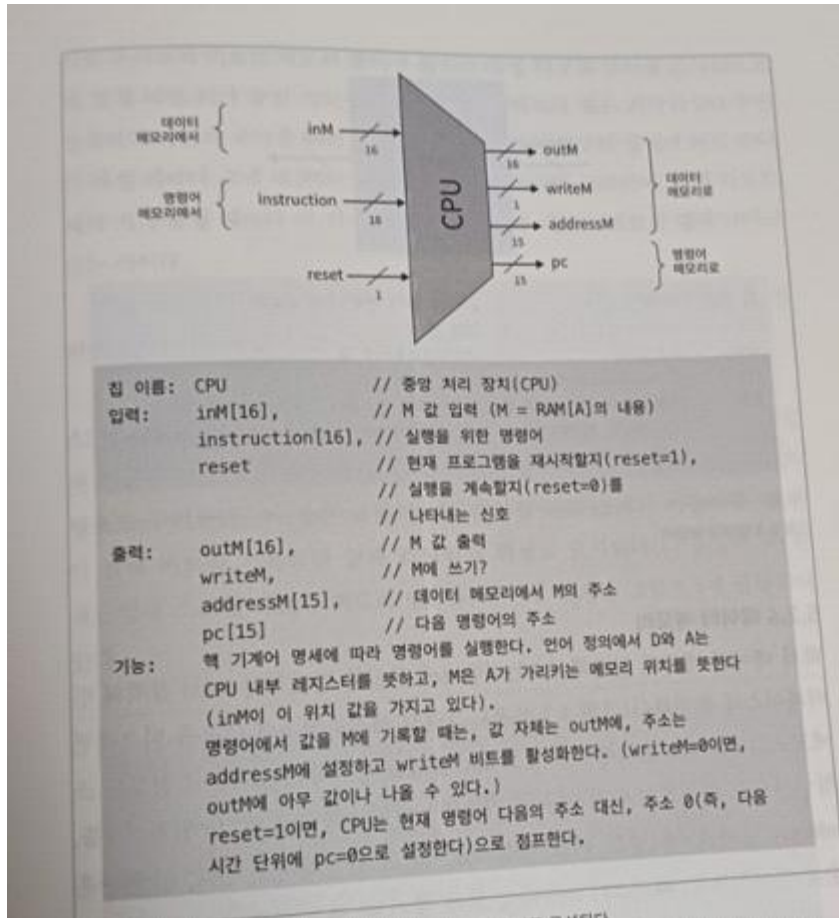
만약 점프가 이루어진다면 PC는 A 레지스터의 값으로 설정

아닌 경우, PC가 1증가한다.

다음 클록 주기에는 PC가 가리키는 명령어가 ROM의 출력이 된다.

>>레지스터와 프로그램 계수기는 16비트 레지스터와 16비트 계수기의 복사본.

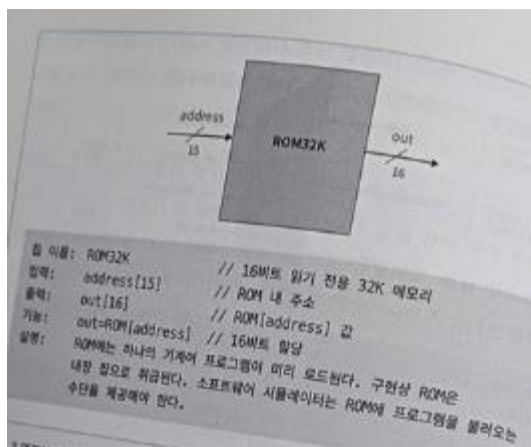
7-2. 중앙 처리 장치 사양



7-3. 명령어 메모리

핵 명령어 메모리는 ROM에 구현.

ROM은 주소 지정 가능한 32K개의 16비트 레지스터로 구성



7-4. 데이터 메모리

데이터 메모리는 컴퓨터의 범용 데이터 저장소 역할 외에도, 메모리 맵을 통한 CPU와 입력/출력 장치 사이의 통신도 맡는다.

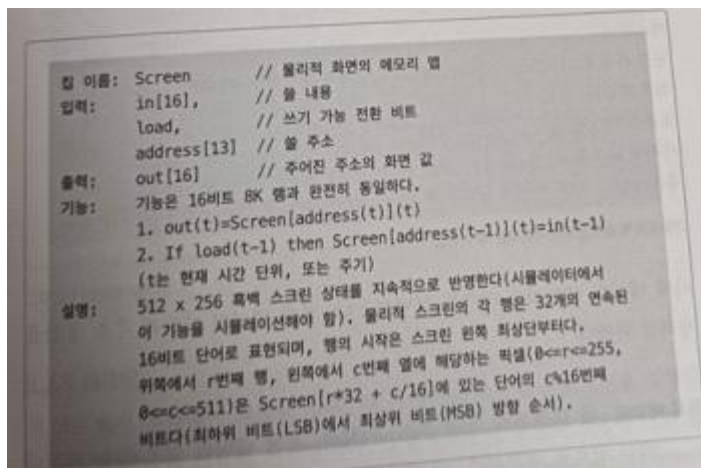
*메모리 맵

스크린과 키보드는 메모리 매핑 버퍼를 통해 컴퓨터 플랫폼과 통신한다.

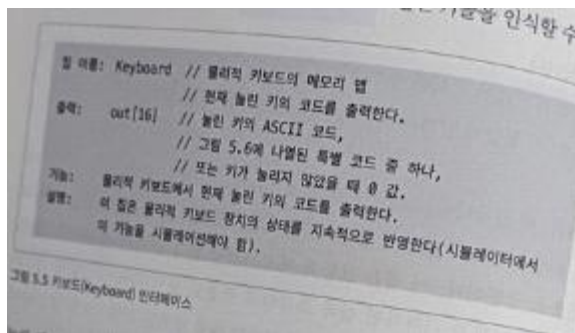
스크린 메모리 맵이라 불리는 특정 메모리 세그먼트에 단어를 읽고 쓰면, 스크린 이미지를 확인하거나 그릴 수 있다.

키보드 메모리 맵이라 불리는 특정 메모리 단어를 조사함으로써 현재 어떤 키가 눌린 상태인지 알 수 있다. (스크린 메모리 맵도 있다)

*스크린



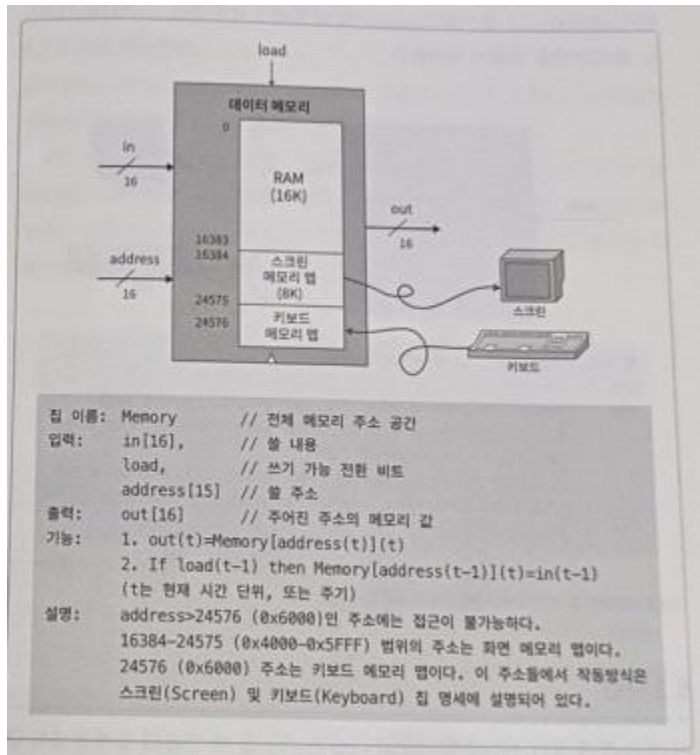
*키보드



키보드에서 키가 눌릴 때마다, 그 키에 대응하는 16비트 ASCII코드가 키보드 칩의 출력으로 나온다.

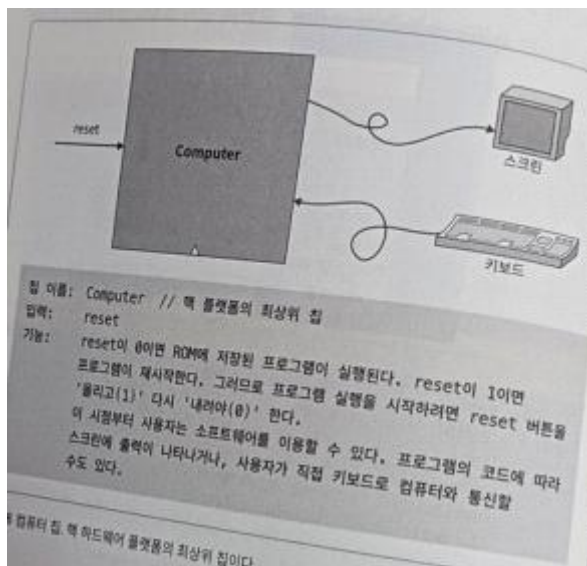
*전체 메모리

핵 플랫폼의 전체 데이터 메모리는 아래와 같다.

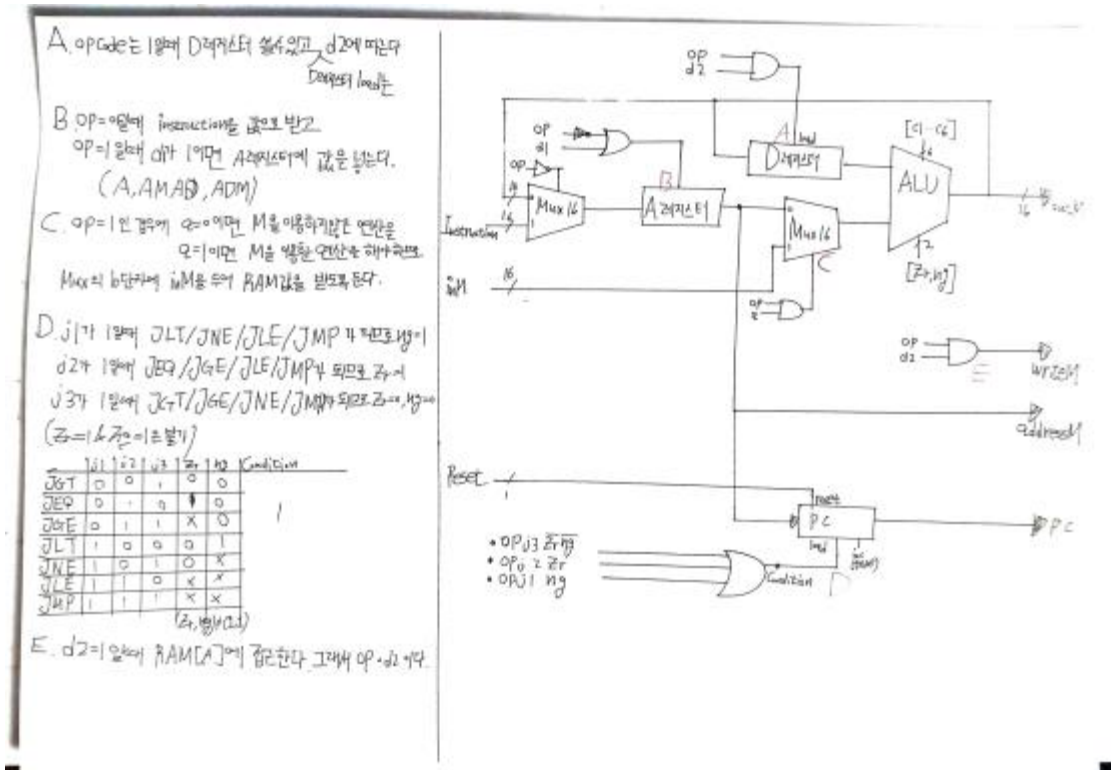


*컴퓨터

컴퓨터 칩은 CPU, 데이터 메모리, 명령어 메모리(ROM), 스크린, 키보드 및 그 외 컴퓨터 작동에 필요한 모든 하드웨어 장치들로 구성된다.



컴퓨터 하드웨어를 설계할 때는 보통 하드웨어 성능을 올리는 데 많은 노력과 창의력이 들어간다. 그래서 하드웨어 아키텍처 강의나 교과서에서는 메모리계층의 활용, IO장치 접근방법의 개선, 파이프라인, 병렬화, 명령어 예비 인출 개념 등의 최적화 기법들을 논한다.



*PC의 inc가 상시 high인 이유는 inc Reset>Load>Inc 순으로 우선순위가 있기 때문이다.

*Instruction이 들어가는 MUX를 보면,

A-명령어인 경우(OP=0) Instruction이 A레지스터에 들어간다.

C-명령어인 경우(OP=1) outM이 A레지스터에 선택적으로 들어가고, (d1에 따라서) Instruction은 값의 의미가 아닌 회로도의 모듈의 제어신호로써 사용된다.

A instruction Symbolic: @xxx (xxx is a decimal value ranging from 0 to 32767, or a symbol bound to such a decimal value)

Binary: 0 vvvvvvvvvvvvvvv (vv ... v = 15-bit value of xxx)

C instruction Symbolic: dest = comp; jump (comp is mandatory.
If dest is empty, the = is omitted;
If jump is empty, the ; is omitted)

Binary: 111 a ccccc c d d j j j

comp	c	c	c	c	c	c	dest	d	d	d	Effect: store comp in:
0							null	0	0	0	the value is not stored
1							M	0	0	1	RAM[A]
-1							D	0	1	0	D register (reg)
D							DM	0	1	1	RAM[A] and D reg
A	M						A	1	0	0	A reg
!D							AM	1	0	1	A reg and RAM[A]
!A	!M						AD	1	1	0	A reg and D reg
-D							ADM	1	1	1	A reg, D reg, and RAM[A]
-A	-M										
D+1							jump	j	j	j	Effect:
A+1	M+1						null	0	0	0	no jump
D-1							JGT	0	0	1	if comp > 0 jump
A-1	M-1						JEQ	0	1	0	if comp = 0 jump
D+A	D+M						JGE	0	1	1	if comp ≥ 0 jump
D-A	D-M						JLT	1	0	0	if comp < 0 jump
A-D	M-D						JNE	1	0	1	if comp ≠ 0 jump
D&A	D&M						JLE	1	1	0	if comp ≤ 0 jump
D A	D M						JMP	1	1	1	unconditional jump

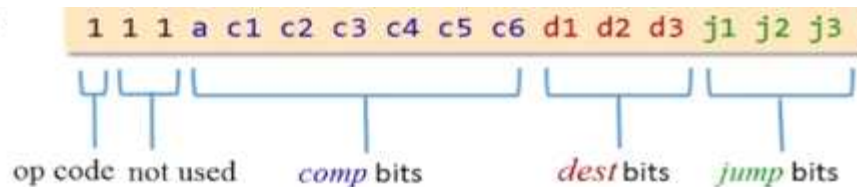
a == 0 a == 1

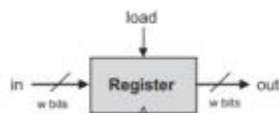
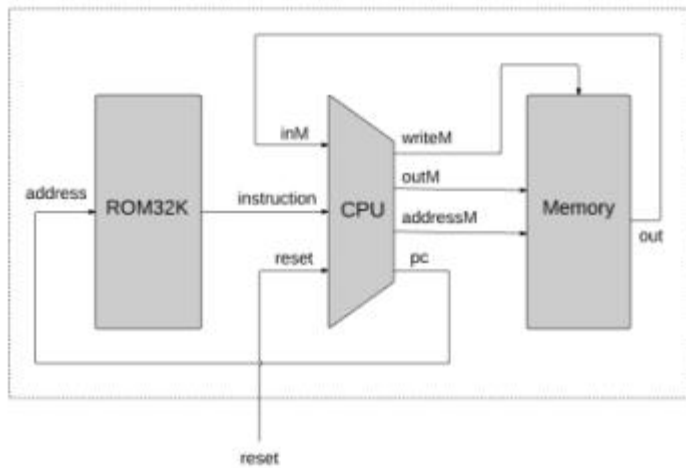
Condition은 PC의 load로 연결한다.

	j1	j2	j3	zr	ng	Condition
null	0	0	0	0	0	0
JGT	0	0	1	0	0	1
JEQ	0	1	0	1	0	1
JGE	0	1	1	x	0	1
JLT	1	0	0	0	0	1
JNE	1	0	1	0	x	1
JLE	1	1	0	x	x	1
JMP	1	1	1	x	x	1

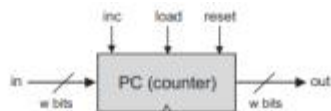
op j1 ng 일때 Condition 1 >>> 3가지 조건일 때 PC는 load는 high 이다
op j2 zr 일때 Condition 1 (zr과 ng 둘다 1는 불가능하다)
op j3 ng| zr 일때 Condition 1

Binary syntax:





Chip name: Register
Inputs: in[16], load
Outputs: out[16]
Function: If load(t-1) then out(t)=in(t-1)
 else out(t)=out(t-1)
Comment: "=" is a 16-bit operation.



Chip name: PC // 16-bit counter
Inputs: in[16], inc, load, reset
Outputs: out[16]
Function: If reset(t-1) then out(t)=0
 else if load(t-1) then out(t)=in(t-1)
 else if inc(t-1) then out(t)=out(t-1)+1
 else out(t)=out(t-1)
Comment: "=" is 16-bit assignment.
 "+" is 16-bit arithmetic addition.

